



**SRI RAMAKRISHNA
ENGINEERING COLLEGE**

[Educational Service : SNR Sons Charitable Trust]

Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.

 www.srec.ac.in

 +917530089996



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACTIVE LEARNING METHODOLOGY REPORT

DC MOTOR SPEED CONTROL USING PWM IN LPC2148

71812401022 – AVANTHIKA K

71812401023 – BALA SURYA S

71812401024 – BALAHARIHARAN A

71812401025 – BHARANISHKUMAR J T

71812401026 – BHARATHI LAKSHMI S

THIRD YEAR B.E. CSE – V SEM

Academic Year 2026-2027

20EC252 & MICROCONTROLLERS AND PROGRAMMING

COURSE COORDINATOR

Mrs.M.Amuthasurabi, AP/CSE



**SRI RAMAKRISHNA
ENGINEERING COLLEGE**

[Educational Service : SNR Sons Charitable Trust]

Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.

 www.srec.ac.in

 +917530089996



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

20EC252 & MICROCONTROLLERS AND PROGRAMMING

Assessment

INDEX

S. No.	Name & Roll No.	Tool Explanation (2)	Implementation (5)	Presentation (3)	Total (10)
1.	71812401022 – AVANTHIKA K				
2.	71812401023 – BALA SURYA S				
3.	71812401024 – BALAHARIHARAN A				
4.	71812401025 – BHARANISHKUMAR J T				
5.	71812401026 – BHARATHI LAKSHMI S				

SIGNATURE OF COURSE COORDINATOR

Mrs.M.Amuthasurabi, AP/CSE



SRI RAMAKRISHNA ENGINEERING COLLEGE

[Educational Service : SNR Sons Charitable Trust]

Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.

 www.srec.ac.in

 +917530089996



CONTENTS

S. No.	TITLE	PAGE NUMBER
1.	INTRODUCTION	4
2.	WORKFLOW DESCRIPTION	5
3.	IMPLEMENTATION STEPS	6
4.	CODE IMPLEMENTATION	8
5.	OUTPUT	12
6.	DIAGRAMMATIC REPRESENTATION	16
7.	CONCLUSION	17

1. INTRODUCTION

Direct-current (DC) motors are widely used in industrial automation to drive conveyors, pumps, fans, robotic joints and machine tools, because their speed can be varied smoothly over a wide range. The speed of a DC motor is nearly proportional to the average voltage applied to its armature, so speed control means controlling that voltage. Reducing the voltage with a series resistor or a linear regulator wastes power as heat and weakens the torque at low speed, whereas a switching method gives precise control with very little loss.

Pulse Width Modulation (PWM) is such a switching method. The supply is switched fully on and fully off at a fixed, high frequency, and the fraction of each period for which it is on, called the duty cycle, sets the average voltage: $V_{avg} = D \times V_s$, where $D = T_{on} / T$. A duty cycle of 25 % on a 12 V supply gives an average of 3 V, and 100 % gives the full 12 V. The inductance and inertia of the motor smooth the pulses, so it runs at a steady speed decided by D. Because PWM is a digital signal, it is generated directly by the microcontroller without a DAC and is exactly repeatable, which is the precise control that an automation system requires.

In this ALM activity the speed control is implemented using the PWM module of the LPC2148, a 32-bit microcontroller from NXP based on the ARM7TDMI-S core. Its main features are listed in Table 1. The PWM module is built around a 32-bit timer with a prescaler and seven match registers, so the period and the on-time of the output are fixed by hardware and the CPU is free while the pulses are being produced.

Table 1: Relevant specifications of the LPC2148

Feature	Specification
Processor core	32-bit ARM7TDMI-S RISC core with ARM and Thumb instruction sets
Clock	Up to 60 MHz CCLK using the on-chip PLL (12 MHz crystal in this design)
On-chip memory	512 kB flash program memory; 32 kB static RAM plus 8 kB RAM shared with USB
PWM module	One PWM block: 32-bit timer/counter with 32-bit prescaler and seven match registers (PWMMR0 to PWMMR6); six single-edge or three double-edge controlled outputs
PWM pins	PWM1 to PWM6 on P0.0, P0.7, P0.1, P0.8, P0.21 and P0.9
Timers / UART	Two 32-bit timers (Timer0, Timer1); two 16C550-compatible UARTs (UART0, UART1)
Interrupts	Vectored Interrupt Controller (VIC) with 16 vectored IRQ slots
GPIO / supply	Up to 45 general-purpose I/O pins; single 3.3 V supply, 3.3 V logic levels

The proposed system uses three on-chip modules of the LPC2148 working together. The PWM module generates a 10 kHz PWM signal on the PWM2 pin (P0.7), which is applied to the enable input of an L293D motor driver, so the duty cycle decides the motor speed. Timer0 raises an interrupt every two seconds so that the duty cycle is stepped through a speed table (20 %, 40 %, 60 %, 80 %, 100 % and back). UART0 sends the duty cycle, the average voltage and the speed to a PC. The complete program is written in Embedded C and built with Keil μ Vision; the generated HEX file is loaded into a Proteus simulation of the LPC2148 with a virtual terminal.

Problem statement: “An industrial automation system requires precise motor control using PWM signals. Explain how the PWM (Pulse Width Modulation) module in the LPC2148 can be used to control the speed of a DC motor. Include the configuration steps and sample pseudo code.”

Objective. The objective of this report is to design and implement the DC motor speed controller so that it can:

- generate a 10 kHz PWM signal on pin P0.7 using the PWM module of the LPC2148;

- change the motor speed by varying the duty cycle between 20 % and 100 %;
- step the speed automatically at a fixed two-second interval using Timer0;
- report the duty cycle, average voltage and speed to a PC through UART0 at 9600 baud; and
- build the program in Keil μ Vision and verify the operation in a simulation.

2. WORKFLOW DESCRIPTION

The speed controller works as a continuous select–update–drive cycle that is paced by Timer0. The complete workflow is shown in Figure 1; each numbered stage is explained below.

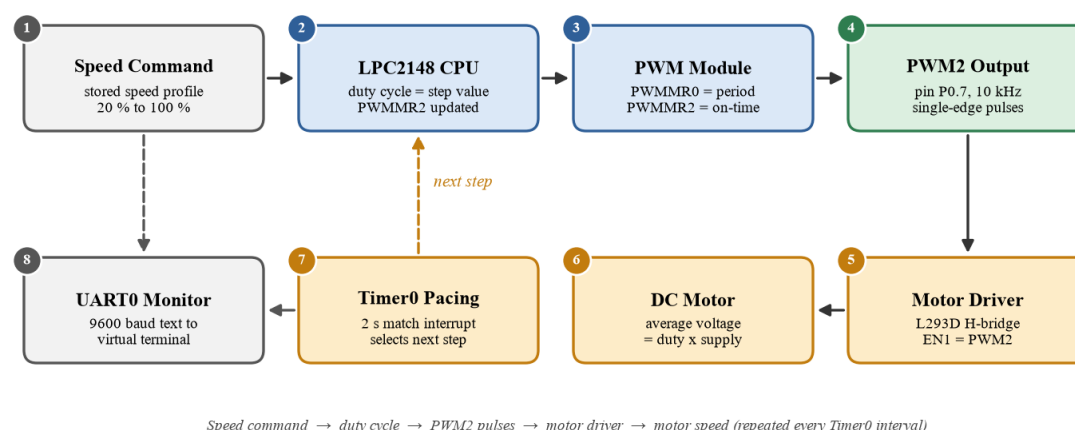


Figure 1: Overall Workflow of the DC Motor Speed Control Using LPC2148 PWM

1. **Speed command.** The required speed is held in the program as a table of duty cycles (20, 40, 60, 80 and 100 %). In a real plant the command could come from a potentiometer or a PLC.
2. **LPC2148 CPU.** The CPU takes the next value from the table and decides the new duty cycle. It writes the value to the match register PWMMR2 and latches it with PWMLER.
3. **PWM module.** The PWM timer counts microseconds from the prescaled PCLK. PWMMR0 fixes the period (100 μ s) and PWMMR2 fixes the on-time, so the module generates the pulses without CPU help.
4. **PWM2 output.** In single-edge mode the output is set at the start of every period and cleared when the counter matches PWMMR2. The signal appears on pin P0.7 at 10 kHz.
5. **Motor driver.** The 3.3 V PWM signal drives the enable pin of an L293D H-bridge. P0.4 and P0.5 set the direction, and the driver switches the 12 V motor supply at the PWM rate.
6. **DC motor.** The motor sees an average voltage of $\text{duty} \times 12 \text{ V}$ and runs at a speed proportional to it. A higher duty cycle gives a higher speed.
7. **Timer0 pacing.** Timer0 is programmed to match every two seconds and raises an interrupt. The interrupt routine only sets a flag; the main program then moves to the next speed step.
8. **UART0 monitor.** The step number, duty cycle, average voltage and speed are sent as text at 9600 baud to a PC (or the virtual terminal), and the cycle repeats.

Table 2: Signal and pin mapping

Signal	LPC2148 pin	Connected to	Function
PWM2	P0.7	L293D EN1	10 kHz PWM, duty = speed
Direction 1	P0.4 (GPIO)	L293D IN1	IN1 = 1 for forward
Direction 2	P0.5 (GPIO)	L293D IN2	IN2 = 0 for forward
TXD0	P0.0	Virtual terminal RXD	Status text, 9600 baud, 8N1

3. IMPLEMENTATION STEPS

The implementation of the speed controller is carried out in Keil μ Vision for program development and compilation. The workflow covers creating the LPC2148 project, selecting the device, adding the source file, configuring the target, building the project, generating the HEX file and finally using the HEX file in the Proteus simulation. The steps are summarised in Table 3 and described in Sections 3.1 to 3.14; the screenshots for these steps are given in Section 5 (Figures 2 to 12).

Table 3: Summary of implementation steps

Step	Activity	Tool / menu
3.1	Open Keil μ Vision	μ Vision 5 (MDK-ARM)
3.2	Create a new project	Project $\rightarrow$ New μ Vision Project
3.3	Select the LPC2148 device	Select Device for Target
3.4	Add the main.c source file	Source Group 1 $\rightarrow$ Add New Item
3.5	Write the Embedded C program	μ Vision editor
3.6	Configure target options	Project $\rightarrow$ Options for Target $\rightarrow$ Target
3.7	Build the project	Project $\rightarrow$ Build Target (F7)
3.8	Generate the HEX file	Options for Target $\rightarrow$ Output
3.9	Configure the simulation	Proteus ISIS, LPC2148 properties
3.10	Connect the motor driver and motor	L293D on P0.7, P0.4, P0.5
3.11	Configure UART / virtual terminal	Virtual Terminal on P0.0 / P0.1
3.12	Run the simulation	Proteus Play button
3.13	Observe the output	Virtual Terminal and motor
3.14	Project completion	Project files and HEX file

3.1 Open Keil μ Vision

The installed Keil μ Vision 5 development environment is opened. The MDK-ARM edition is used because the LPC2148 is an ARM7-based microcontroller. The Project window, the editor area and the Build Output window are displayed.

3.2 Create a New Project

A new project is created using Project $\rightarrow$ New μ Vision Project. A suitable project name such as DC_Motor_PWM is entered, and a separate folder is selected for the project files (Figure 2). Keeping each project in its own folder keeps the source, object and HEX files organised.

3.3 Select LPC2148

After the project is saved, the Select Device for Target window opens. LPC2148 is searched and selected under the NXP (founded by Philips) LPC2000 family, and OK is clicked (Figure 3). When μ Vision asks whether to copy the startup file Startup.s to the project folder and add it to the project, Yes is chosen; Startup.s initialises the stacks and the vector table.

3.4 Add main.c

In the Project window, Source Group 1 is right-clicked and Add New Item to Group 'Source Group 1' is chosen. A C File (.c) is selected and saved with the name main (Figure 4). This file contains the complete Embedded C program for the PWM, UART0 and Timer0.

3.5 Write the Embedded C Program

The program of Section 4 is typed into main.c and saved with Ctrl + S (Figure 5). It includes the header file LPC214x.h to access the LPC2148 registers, configures the PLL, GPIO, PWM, UART0 and Timer0, and contains the Timer0 interrupt routine, the duty-cycle update routine and the main loop.

3.6 Configure Target Options

Project → Options for Target is opened (Alt + F7). On the Target tab the device is verified and the crystal frequency is entered as 12.000 MHz, matching the hardware assumption (Figure 6). The memory areas selected for the LPC2148 are accepted, and Use MicroLIB may be ticked to reduce the program size.

3.7 Build the Project

The project is compiled with Project → Build Target or by pressing F7. µVision compiles main.c and Startup.s, links the output and checks the program for errors. The Build Output window shows the program size and the numbers of errors and warnings (Figure 7). A successful build should report 0 errors.

3.8 Generate HEX File

To create the file needed by the simulation, Options for Target → Output is opened and Create HEX File is ticked (Figure 8). After clicking OK, the project is rebuilt and µVision generates DC_Motor_PWM.hex in the Objects folder in Intel HEX format.

3.9 Configure Simulation

The simulation is carried out in Proteus. An LPC2148 is placed in the schematic together with an L293D, a DC motor and a virtual terminal (Figure 9). In the properties of the LPC2148 the Program File is set to the HEX file generated by Keil and the crystal frequency to 12 MHz.

3.10 Connect Motor Driver and Motor

P0.7 (PWM2) is connected to EN1 of the L293D, P0.4 to IN1 and P0.5 to IN2, as listed in Table 4. The motor is connected to OUT1 and OUT2, and the motor supply pin VS is connected to +12 V. All grounds are joined.

Table 4: Connections used for the simulation

Proteus component	Connected to	Represents / setting
L293D EN1	P0.7 (PWM2)	PWM speed input, 10 kHz
L293D IN1 / IN2	P0.4 / P0.5	Direction inputs (1 / 0 = forward)
L293D OUT1 / OUT2	DC motor terminals	Motor driven by the 12 V supply (VS)
Virtual Terminal RXD	P0.0 (TXD0)	Receives the data sent by UART0
Virtual Terminal TXD	P0.1 (RXD0)	Connected for completeness; not used

3.11 Configure UART / Virtual Terminal

The Virtual Terminal is connected to UART0 and its properties are set to match the program: baud rate 9600, 8 data bits, no parity and 1 stop bit. The terminal window opens automatically when the simulation starts.

3.12 Run the Simulation

After the HEX file is loaded and the connections are checked, the Play button in Proteus is pressed. The LPC2148 initialises the PLL, GPIO, PWM, UART0 and Timer0, sends the start-up banner, and from then on Timer0 changes the duty cycle every two seconds.

3.13 Observe the Output

The Virtual Terminal first shows the banner DC MOTOR SPEED CONTROL USING PWM. After every timer interval a block is added containing the step number, the duty cycle, the average voltage and the speed (Figures 10 to 12). The motor speed rises and falls with the duty cycle.

3.14 Project Completion

After successful compilation and simulation, the implementation is complete. The project contains the LPC2148 target, the Embedded C source file main.c, the generated HEX file and the Proteus simulation.

4. CODE IMPLEMENTATION

The software of the speed controller is developed in Embedded C using Keil μ Vision. The program is written specifically for the LPC2148 and uses the header file LPC214x.h for access to the special function registers. It assumes a 12 MHz crystal, CCLK = 60 MHz and PCLK = 15 MHz. Sections 4.1 to 4.7 describe the individual parts of the program and Section 4.8 gives the complete program.

4.1 Pin Configuration

Table 5: Pin configuration of the DC motor speed controller

Function	LPC2148 pin	Purpose
PWM output	P0.7 / PWM2	10 kHz PWM to the enable pin of the L293D
Motor direction 1	P0.4 (GPIO)	IN1 of the L293D, set to 1
Motor direction 2	P0.5 (GPIO)	IN2 of the L293D, set to 0
UART TX	P0.0 / TXD0	Transmits the status text to the PC
UART RX	P0.1 / RXD0	Receive line of UART0 (configured, not used)

4.2 PWM Configuration Steps

The PWM module is configured with the following steps; the values used are collected in Table 6.

1. **Select the pin function.** PINSEL0 bits 15:14 are set to 10 so that P0.7 becomes the PWM2 output.
2. **Set the clock.** The PLL gives CCLK = 60 MHz and VPBDIV = 0 gives PCLK = CCLK / 4 = 15 MHz.
3. **Reset the PWM timer.** PWMTCCR = 0x02 holds the counter in reset while it is configured.
4. **Set the prescaler.** PWMPR = 14 makes the counter tick every $(14 + 1) / 15 \text{ MHz} = 1 \mu\text{s}$.
5. **Set the period.** PWMMR0 = 100 gives a period of 100 μs , that is a PWM frequency of 10 kHz.
6. **Set the duty cycle.** PWMMR2 = duty value (0 to 100) fixes the on-time, so duty (%) = $\text{PWMMR2} / \text{PWMMR0} \times 100$.
7. **Choose the match action.** PWMMCR = 0x02 resets the counter when it matches PWMMR0.
8. **Latch the values.** PWMLER = 0x05 loads the new PWMMR0 and PWMMR2 at the next period.
9. **Enable the output.** PWMPCR bit 10 (PWMENA2) = 1 enables PWM2 in single-edge mode.
10. **Start the PWM.** PWMTCCR = 0x09 enables the counter (bit 0) and the PWM mode (bit 3).

Table 6: PWM configuration parameters

Parameter	Register / bits	Value used
Pin function	PINSEL0 bits 15:14	10 (P0.7 = PWM2)
PWM timer clock	PWMPR	14 $\rightarrow$ 1 μs per tick (PCLK = 15 MHz)
Period (frequency)	PWMMR0	100 $\rightarrow$ 100 μs (10 kHz)
Duty cycle	PWMMR2	20, 40, 60, 80, 100 (% of period)
Match control	PWMMCR	0x02 (reset counter on MR0)
Latch enable	PWMLER	0x05 at start, 0x04 for later duty updates
Output enable / mode	PWMPCR bit 10	1 (PWMENA2, single-edge)
Timer control	PWMTCCR	0x02 reset, then 0x09 (counter + PWM enable)

4.3 Pseudo Code

START

```

Configure PLL: CCLK = 60 MHz, set PCLK = CCLK / 4 = 15 MHz
Set P0.7 as PWM2 output (PINSEL0); set P0.4 and P0.5 as GPIO outputs
Set direction: IN1 = 1, IN2 = 0                                (forward rotation)
Initialise UART0 (9600 baud, 8N1); Initialise Timer0 (interrupt every 2 s)
PWM CONFIGURATION
    PWMTCCR <- reset counter
    PWMPR <- 14                                                (1 tick = 1 us)
    PWMMR0 <- 100                                              (period = 100 us, 10 kHz)
    PWMMR2 <- initial duty (20)                                (on-time = duty x 1 us)
    PWMMCR <- reset counter on MR0 match
    PWMLER <- latch MR0 and MR2
    PWMPCR <- enable PWM2 output, single-edge
    PWMTCCR <- enable counter and PWM mode
index <- 0
LOOP forever
    IF step_flag = 1 THEN
        step_flag <- 0
        duty <- speed_table[index]
        PWMMR2 <- duty; latch MR2 through PWMLER
        voltage <- duty x 12 V / 100; speed <- duty x 30 RPM
        send step number, duty, voltage, speed through UART0
        index <- (index + 1) MOD table size
    END IF
END LOOP
TIMER0 ISR: step_flag <- 1; clear Timer0 interrupt; signal end of interrupt
END

```

4.4 PWM Initialization and Duty Update

`pwm_init()` performs the configuration steps of Section 4.2. `pwm_set_duty()` writes a new value to `PWMMR2` and latches it with `PWMLER`; the new duty cycle takes effect at the start of the next period, so the output never produces a shortened or glitched pulse. For 100 % the match value is set above the period so that it never matches and the output stays high.

```

void pwm_init(void)
{
    PINSEL0 = (PINSEL0 & ~(3u << 14)) | (2u << 14); /* P0.7 = PWM2 */
    PWMTCCR = 0x02; /* reset the counter */
    PWMPR = (PCLK_HZ / 1000000UL) - 1; /* 14 -> 1 us per tick */
    PWMMR0 = PWM_PERIOD; /* 100 us period -> 10 kHz */
    PWMMR2 = speed_table[0]; /* initial duty 20 % */
    PWMMCR = 0x02; /* reset counter on MR0 match */
    PWMLER = 0x05; /* latch MR0 and MR2 */
    PWMPCR = (1u << 10); /* PWMENA2, single-edge */
    PWMTCCR = 0x09; /* counter enable + PWM enable */
}

void pwm_set_duty(unsigned int duty)
{
    if (duty >= 100) PWMMR2 = PWM_PERIOD + 1; /* never matches: always high */
    else PWMMR2 = duty; /* on-time = duty x 1 us */
    PWMLER = 0x04; /* latch MR2 at next period */
}

```

4.5 UART0 Initialization and Transmission

UART0 is configured for 9600 baud, 8 data bits, no parity and 1 stop bit. `PINSEL0` bits 3:0 are set to 0101 so that P0.0 is TXD0 and P0.1 is RXD0. With `DLAB = 1`, the divisor $PCLK / (16 \times 9600) = 97.7$, rounded to 98, is written to `U0DLL` (actual baud rate 9566, error 0.35 %). `uart0_putc()` waits until `THRE = 1` in `U0LSR` and writes the character to `U0THR`; `uart0_puts()` sends a complete string.

4.6 Timer0 Initialization and Interrupt

`T0PR = 14999` makes Timer0 tick every 1 ms and `T0MR0 = 1999` produces a match after 2000 ticks, which is 2 s. `T0MCR = 0x03` generates an interrupt and resets the counter at every match. Timer0 is source 4 of the VIC and uses vectored slot 0. The interrupt routine only sets `step_flag`, clears the MR0 flag by writing 1 to `T0IR` and writes 0 to `VICVectAddr`; the keyword `__irq` makes the ARM compiler generate the interrupt entry and exit code.

4.7 Data Processing and Report

report() converts the duty cycle into the average motor voltage (tenths of a volt = $\text{duty} \times 12 / 10$) and into an approximate no-load speed ($\text{duty} \times 30$ RPM, assuming a 3000 RPM motor at full voltage), using integer arithmetic only. The values are formatted with printf() and sent through UART0.

4.8 Complete Program

The complete program initialises the PLL, GPIO, PWM, UART0 and Timer0 in that order, sends the start-up banner and then enters an endless loop. The loop checks the volatile flag step_flag, which Timer0 sets every two seconds; when it is set, the flag is cleared, the next duty cycle is written to the PWM module and the result is reported through UART0.

```
/* DC Motor Speed Control using PWM - LPC2148 - main.c (Keil uVision, Embedded C) */
#include <LPC214x.h>
#include <stdio.h>
#define PCLK_HZ      15000000UL      /* PCLK = CCLK / 4 = 15 MHz */
#define BAUD         9600UL
#define STEP_MS      2000UL         /* speed changes every 2 s */
#define PWM_PERIOD   100            /* PWM0: 100 ticks x 1 us = 100 us */
#define MOTOR_IN1    (1u << 4)     /* P0.4: L293D IN1 */
#define MOTOR_IN2    (1u << 5)     /* P0.5: L293D IN2 */
#define NUM_STEPS    9
const unsigned char speed_table[NUM_STEPS] = {20, 40, 60, 80, 100, 80, 60, 40, 20};
volatile unsigned char step_flag = 0; /* set by the Timer0 interrupt */
unsigned int step_no = 0;
/* ----- clock ----- */
void pll_init(void)
{
    unsigned int t = 100000;          /* lock time-out counter */
    PLL0CFG = 0x24;                   /* M = 5, P = 2 -> CCLK = 60 MHz */
    PLL0CON = 0x01;                   /* enable PLL */
    PLL0FEED = 0xAA; PLL0FEED = 0x55;
    while (!(PLL0STAT & (1u << 10)) && --t); /* wait for PLL lock (time-out) */
    PLL0CON = 0x03;                   /* enable and connect PLL */
    PLL0FEED = 0xAA; PLL0FEED = 0x55;
    MAMTIM = 0x03;                    /* flash accelerator timing */
    MAMCR = 0x02;                     /* fully enabled */
    VPBDIV = 0x00;                    /* PCLK = CCLK / 4 = 15 MHz */
}
/* ----- GPIO ----- */
void gpio_init(void)
{
    IO0DIR |= (MOTOR_IN1 | MOTOR_IN2); /* P0.4, P0.5 as outputs */
    IO0SET = MOTOR_IN1;                /* IN1 = 1 */
    IO0CLR = MOTOR_IN2;                /* IN2 = 0 -> forward */
}
/* ----- PWM ----- */
void pwm_init(void)
{
    PINSEL0 = (PINSEL0 & ~(3u << 14)) | (2u << 14); /* P0.7 = PWM2 */
    PWMTCR = 0x02; /* reset the counter */
    PWMPR = (PCLK_HZ / 1000000UL) - 1; /* 14 -> 1 us per tick */
    PWMMR0 = PWM_PERIOD; /* 100 us period -> 10 kHz */
    PWMMR2 = speed_table[0]; /* initial duty 20 % */
    PWMPCR = 0x02; /* reset counter on MR0 match */
    PWMLER = 0x05; /* latch MR0 and MR2 */
    PWMPCR = (1u << 10); /* PWMENA2, single-edge */
    PWMTCR = 0x09; /* counter enable + PWM enable */
}
void pwm_set_duty(unsigned int duty)
{
    if (duty >= 100) PWMMR2 = PWM_PERIOD + 1; /* never matches: always high */
    else PWMMR2 = duty; /* on-time = duty x 1 us */
    PWMLER = 0x04; /* latch MR2 at next period */
}
/* ----- UART0 ----- */
void uart0_init(void)
{
    PINSEL0 = (PINSEL0 & ~0x0Fu) | 0x05u; /* P0.0 = TXD0, P0.1 = RXD0 */
    U0LCR = 0x83; /* 8N1, DLAB = 1 */
    U0DLL = ((PCLK_HZ / 16) + (BAUD / 2)) / BAUD; /* divisor = 98 */
    U0DLM = 0;
}
```

```

    U0LCR = 0x03;                /* DLAB = 0 */
    U0FCR = 0x07;                /* enable and reset FIFOs */
}
void uart0_putc(char c)
{
    while (!(U0LSR & 0x20));      /* wait until THR is empty */
    U0THR = c;
}
void uart0_puts(const char *s)
{
    while (*s)
        uart0_putc(*s++);
}
/* ----- Timer0 ----- */
__irq void timer0_isr(void)
{
    step_flag = 1;                /* 2 s has elapsed */
    T0IR = 0x01;                  /* clear MR0 interrupt flag */
    VICVectAddr = 0;              /* end of interrupt */
}
void timer0_init(void)
{
    T0TCR = 0x02;                 /* reset counter */
    T0PR = (PCLK_HZ / 1000) - 1;  /* 14999 -> 1 ms per tick */
    T0MR0 = STEP_MS - 1;          /* 1999 -> match every 2 s */
    T0MCR = 0x03;                 /* interrupt + reset on MR0 */
    VICVectAddr0 = (unsigned long)timer0_isr;
    VICVectCntl0 = 0x20 | 4;      /* slot 0 enabled, Timer0 = 4 */
    VICIntEnable = (1u << 4);
    T0TCR = 0x01;                 /* start Timer0 */
}
/* ----- report over UART0 ----- */
void report(unsigned int duty)
{
    char buf[40];
    unsigned int v10 = (duty * 6u) / 5u; /* tenths of a volt (12 V supply) */
    sprintf(buf, "Step No      : %02u\r\n", step_no);   uart0_puts(buf);
    sprintf(buf, "PWM Duty     : %u %%\r\n", duty);     uart0_puts(buf);
    sprintf(buf, "Avg Voltage   : %u.%u V\r\n", v10 / 10, v10 % 10);  uart0_puts(buf);
    sprintf(buf, "Motor Speed   : %u RPM\r\n", duty * 30u);          uart0_puts(buf);
    uart0_puts("Direction     : FORWARD\r\n");
    uart0_puts("-----\r\n");
    uart0_puts("PWM Updated Successfully\r\n\r\n");
}
/* ----- main ----- */
int main(void)
{
    unsigned int idx = 0;
    pll_init();
    gpio_init();
    pwm_init();
    uart0_init();
    timer0_init();
    uart0_puts("=====\r\n");
    uart0_puts("DC MOTOR SPEED CONTROL USING PWM\r\n");
    uart0_puts("=====\r\n");
    while (1)
    {
        if (step_flag)                /* set every 2 s by Timer0 */
        {
            step_flag = 0;
            pwm_set_duty(speed_table[idx]); /* update PWMMR2 and latch */
            step_no++;
            report(speed_table[idx]);
            idx = (idx + 1) % NUM_STEPS;
        }
    }
}

```

5. OUTPUT

This section presents the screenshots of the Keil μ Vision implementation and of the simulation of the DC motor speed controller: project creation, device selection, source file, program, target configuration, successful build, HEX file generation, simulation setup and the UART output received on the virtual terminal. Each figure is followed by an explanation of what it demonstrates.

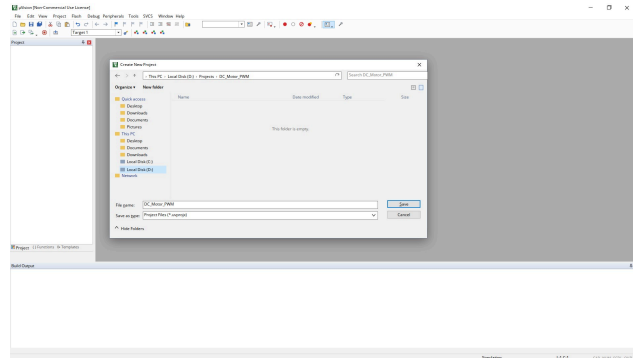


Figure 2: Creating a New uVision Project

Figure 2 shows the new project being created and saved in its own folder.

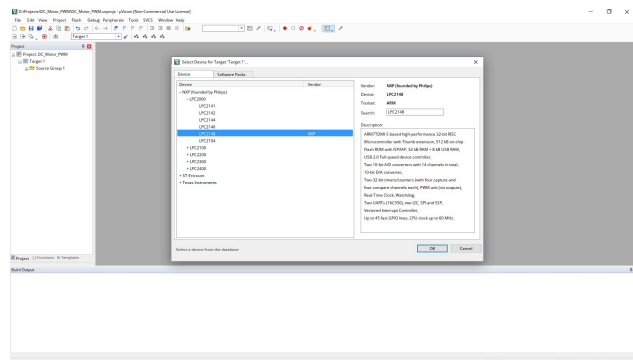


Figure 3: Selecting LPC2148 Device

Figure 3 shows the LPC2148 device selected for the target.

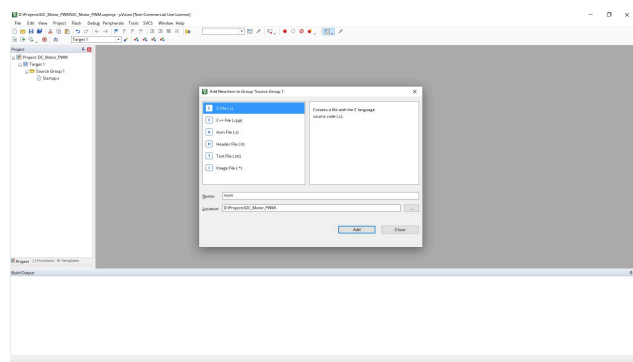


Figure 4: Adding main.c Source File

Figure 4 shows the main.c source file being added to Source Group 1.

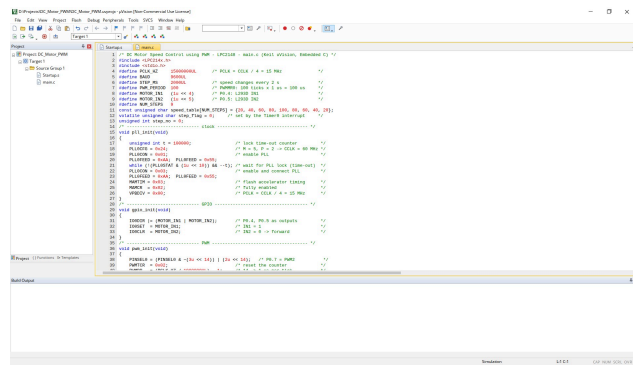


Figure 5: Embedded C Program in Keil uVision

Figure 5 shows the Embedded C program of the DC motor speed controller in the μ Vision editor.

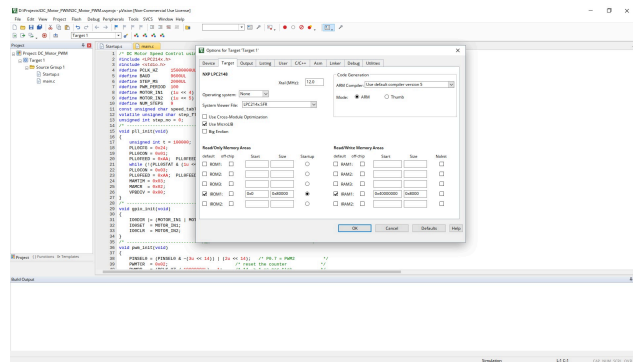


Figure 6: Target Configuration for LPC2148

Figure 6 shows the target options configured for the LPC2148 with a 12 MHz crystal.

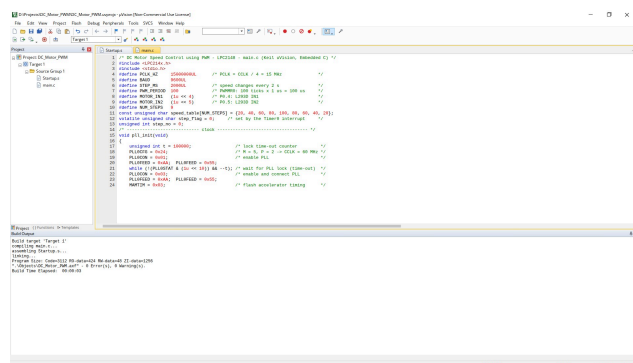


Figure 7: Successful Build Output

Figure 7 shows the build output of the project. A successful compilation reports 0 errors and the program size, confirming that the Embedded C program is correct for the LPC2148.

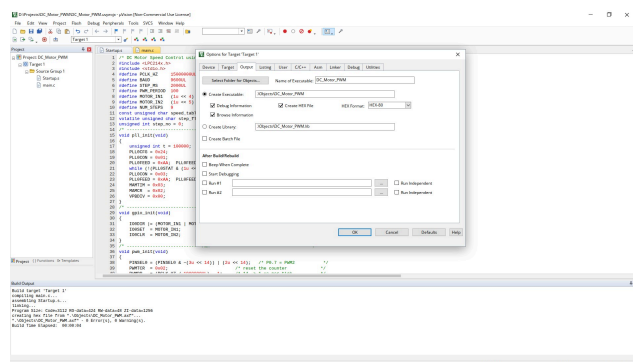


Figure 8: HEX File Generation

Figure 8 shows the Create HEX File option enabled. After rebuilding, the .hex file is generated and loaded into the LPC2148 model in Proteus.

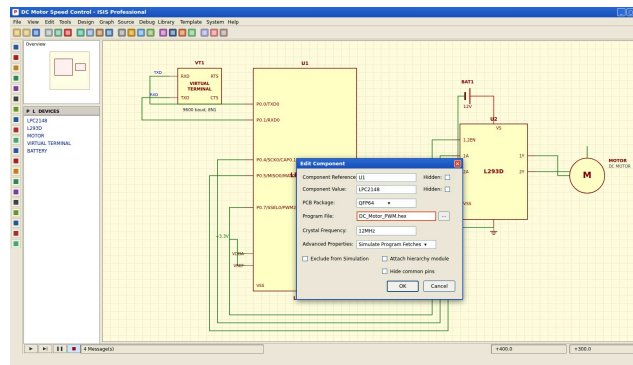


Figure 9: Simulation Setup

Figure 9 shows the simulation circuit: the LPC2148, the L293D motor driver, the DC motor and the virtual terminal connected to UART0.

```

Virtual Terminal (UART0, 9600 baud, 8N1) - calculated output

=====
DC MOTOR SPEED CONTROL USING PWM
=====

Step No      : 01
PWM Duty     : 20 %
Avg Voltage  : 2.4 V
Motor Speed  : 600 RPM
Direction    : FORWARD
-----
PWM Updated Successfully

Step No      : 02
PWM Duty     : 40 %
Avg Voltage  : 4.8 V
Motor Speed  : 1200 RPM
Direction    : FORWARD
-----
PWM Updated Successfully

Step No      : 03
PWM Duty     : 60 %
Avg Voltage  : 7.2 V
Motor Speed  : 1800 RPM
Direction    : FORWARD
-----
PWM Updated Successfully

```

Figure 10: Expected UART Output of the DC Motor Speed Controller (calculated)

Figure 10 shows the data sent through UART0: the banner and the numbered step blocks with the duty cycle, average voltage and speed. The values are calculated from the program (Table 7).

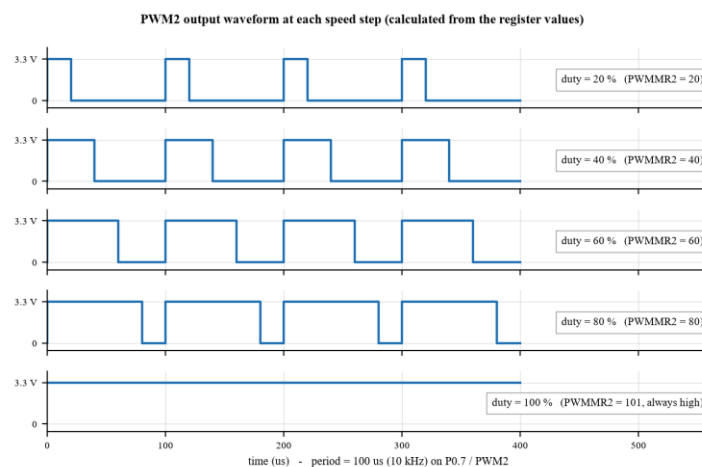


Figure 11: PWM2 Output Waveform at Each Speed Step (calculated)

Figure 11 shows the pulses on P0.7: the period stays at 100 μ s while the on-time grows from 20 μ s to 100 μ s, which raises the average voltage and therefore the motor speed.

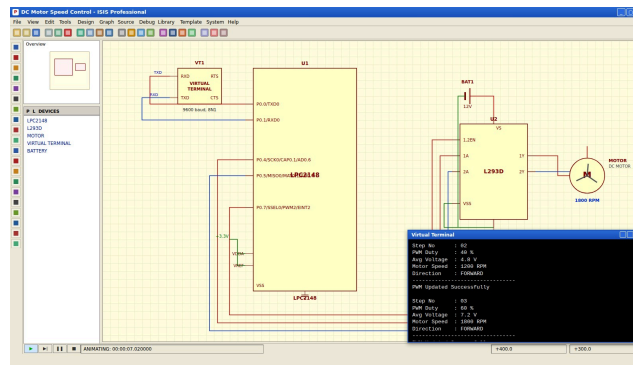


Figure 12: Final Output

Figure 12 shows the complete system running: the LPC2148, the motor driver, the motor and the virtual terminal displaying the speed steps.

Sample output. Table 7 lists the output that the program produces for each speed step. These values are calculated with the formulas of the program (average voltage = duty $\times$ 12 V, speed = duty $\times$ 30 RPM for an assumed 3000 RPM motor) and are given as sample / expected values.

Table 7: Sample output of the speed controller (calculated)

Step	Duty cycle	PWMMR2	Pulse width (Ton)	Avg voltage	Motor speed
01	20 %	20	20 μ s	2.4 V	600 RPM
02	40 %	40	40 μ s	4.8 V	1200 RPM
03	60 %	60	60 μ s	7.2 V	1800 RPM
04	80 %	80	80 μ s	9.6 V	2400 RPM
05	100 %	101	always high	12.0 V	3000 RPM

The output format sent by the program is shown below for the first step. A new block is sent every two seconds, the step number increases by one each time, and the speed values follow the table 20, 40, 60, 80, 100, 80, 60, 40, 20 % before the cycle repeats.

```

=====
DC MOTOR SPEED CONTROL USING PWM
=====
Step No      : 01
PWM Duty     : 20 %
Avg Voltage  : 2.4 V
Motor Speed  : 600 RPM
Direction    : FORWARD
-----
PWM Updated Successfully

```

6. DIAGRAMMATIC REPRESENTATION

6.1 System Block Diagram

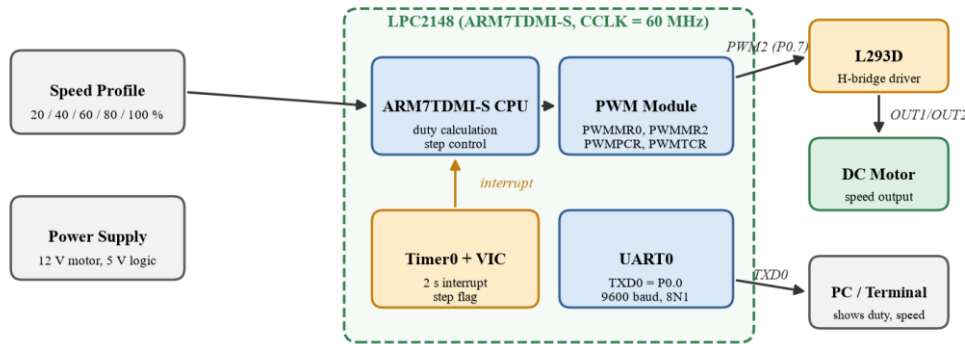


Figure 13: System Block Diagram of the DC Motor Speed Controller

The block diagram shows the speed profile processed by the CPU, the PWM module generating the PWM2 signal, Timer0 stepping the speed through an interrupt, the L293D driving the motor, and UART0 sending the status to the PC.

6.2 PWM Generation Diagram

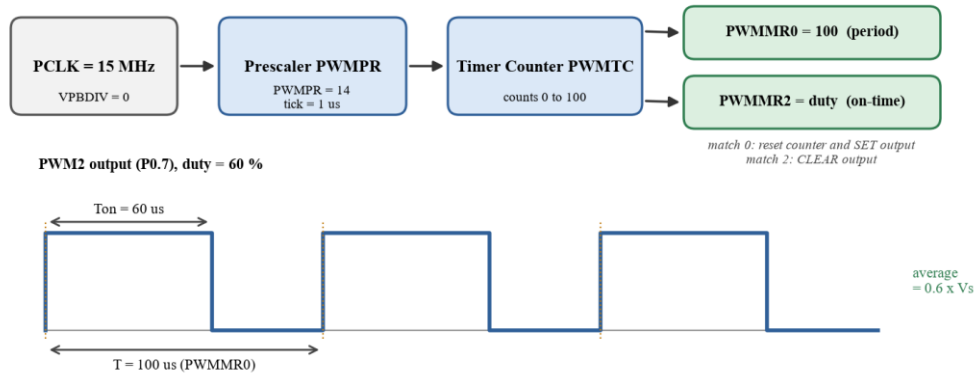


Figure 14: PWM Generation in the LPC2148 (PCLK = 15 MHz, 10 kHz, duty = 60 % shown)

The prescaler divides PCLK to a 1 μ s tick, and the timer counter counts from 0 to PWMMR0 = 100. The output is set when the counter restarts and cleared when it matches PWMMR2, so the on-time equals $\text{PWMMR2} \times 1 \mu\text{s}$ and the average voltage is $\text{duty} \times V_s$.

6.3 Motor Driver Connection Diagram

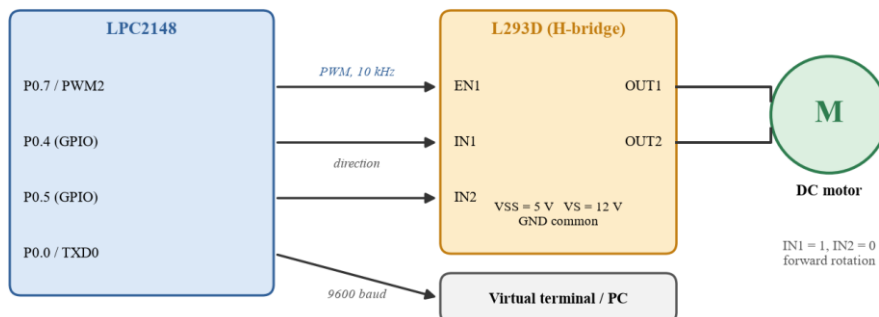


Figure 15: Interface between the LPC2148, the L293D and the DC Motor

PWM2 (P0.7) is connected to the enable pin EN1 of the L293D, and P0.4 and P0.5 are connected to IN1 and IN2 for the direction. The 3.3 V outputs of the LPC2148 are above the L293D input-high threshold, and the motor is powered separately from 12 V with a common ground.

6.4 Timer-Based Speed Control Flowchart

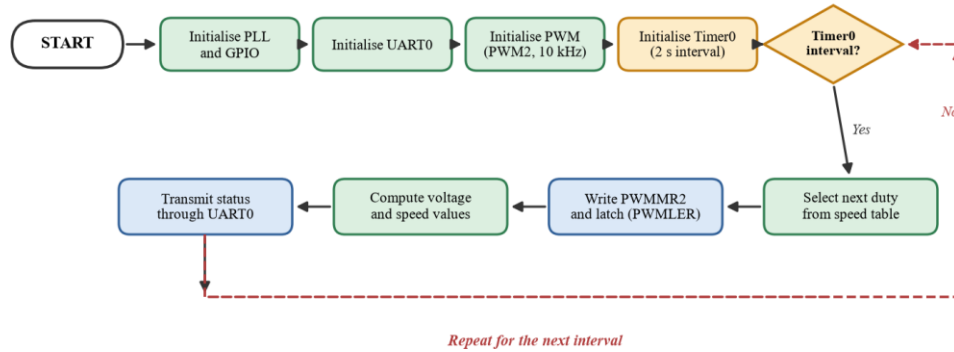


Figure 16: Timer0-Based Speed Control Flowchart of the DC Motor Controller

After the PLL, GPIO, UART, PWM and Timer0 are initialised, the program waits for the Timer0 interval. When it elapses, the next duty cycle is selected and written to PWMMR2, the values are computed and transmitted through UART0, and the program waits for the next interval.

6.5 Overall System Architecture

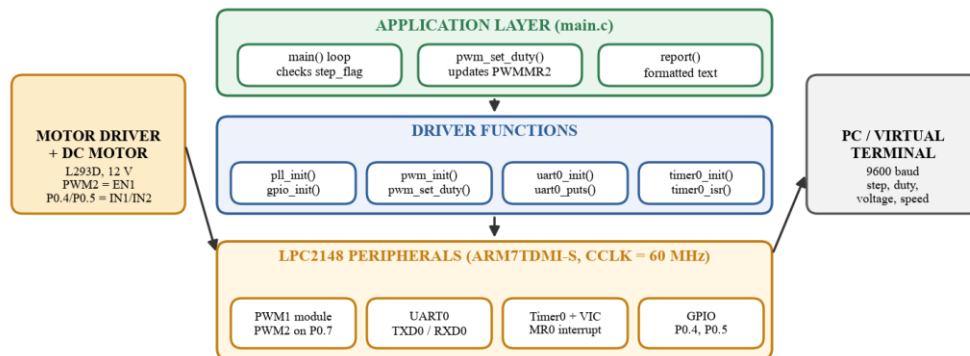


Figure 17: Overall Architecture of the DC Motor Speed Controller

The architecture shows the three layers of the system: the application layer in main.c, the driver functions for the PLL, GPIO, PWM, UART0 and Timer0, and the LPC2148 peripherals that they control. The motor driver is fed from the PWM output and the PC receives the status from UART0.

7. CONCLUSION

A DC motor speed controller was designed and implemented around the PWM module of the LPC2148 microcontroller. The PWM timer, with PWMPR = 14 and PWMMR0 = 100, generates a 10 kHz signal on P0.7 whose duty cycle, set through PWMMR2, fixes the average voltage across the motor and therefore its speed. The speed was stepped from 20 % to 100 % and back, and the status was sent to a PC through UART0 at 9600 baud, 8N1.

The PWM module is essential because it produces accurate, jitter-free pulses in hardware, so the CPU is not occupied with switching the output and the duty cycle can be changed in a single register write. The latch enable register makes every change take effect at a period boundary, and

Timer0 makes the speed steps periodic and independent of the program flow. The integration of the three modules makes the software short, deterministic and easy to extend.

The program was developed in Embedded C using Keil μ Vision: the LPC2148 project was created, main.c was added, the target was configured for a 12 MHz crystal, the project was built and the HEX file was generated for the simulation. Such a controller can be applied to conveyor drives, pumps and fans, robotic joints, and other speed-controlled loads. Possible future improvements are:

- closed-loop control using an encoder or tachogenerator and a PI controller;
- setting the speed from a potentiometer through the on-chip ADC;
- reversing the direction through P0.4 and P0.5 with a soft-start ramp;
- adding current sensing for overload protection; and
- sending the speed commands and receiving feedback through UART0 from a PC or PLC.